PCU-Select: Amortizing Data Selection Across a Registry of PEFT Configurations

Anonymous submission

Abstract

A machine-learning platform often maintains a registry of parameter-efficient fine-tuning (PEFT) configurations to support diverse tasks and deployment budgets. Gradient- and influence-based data selectors can produce strong target-specific subsets, but they require rebuilding a target-configuration-specific gradient datastore, causing selection cost to grow linearly with registry size. Reusing a single subset avoids this cost but sacrifices quality because example utility depends on the trainable subspace. We present PCU-Select, which amortizes data selection across a PEFT registry by conditioning utility on both the task and a structured PEFT code. Its key design is a set of 24 shared intervention sites that represent heterogeneous PEFT methods in common module-output coordinates, allowing sample and task signatures to be reused across configurations. Offline, PCU-Select combines broad low-fidelity gradient proxies with high-fidelity labels from short PEFT adaptation runs to train a conditional scorer that predicts utility and uncertainty. At selection time, the scorer ranks cached candidate features for a target PEFT–task pair without rebuilding a gradient datastore. On Llama-2-7B across four tasks and five seen PEFTs, PCU-Select achieves 35.18 average quality, nearly tying per-PEFT LESS at 35.14. Yet onboarding a new four-task PEFT costs only 1.44 GPU-hours instead of 63.2, a 44 times reduction. By turning per-configuration recomputation into once-offline amortization, PCU-Select makes gradient-level data selection practical at registry scale.

1 Introduction

Efficient adaptation of large language models requires reducing both the number of trainable parameters and the amount of training data. A machine-learning platform must also serve heterogeneous tasks and deployment constraints, and therefore rarely relies on a single adaptation setting. Instead, it keeps a *registry* of PEFT configurations for different tasks, latency targets, and memory limits. The registry may include LoRA at several ranks and placements, adapters, $(IA)^3$, prefix tuning, and BitFit. Data-efficient fine-tuning complements these methods by selecting a compact subset from a larger instruction pool. In current practice, however, PEFT controls which parameters move while data selection independently chooses which examples drive the update.

Current data selectors rarely model this interaction. Quality filters, diversity heuristics, and proxy-model scores assign each example one value for a target task, then reuse the selected subset across adaptation methods. This PEFT-agnostic assumption ignores how each method changes the trainable subspace. LoRA injects low-rank additive updates into selected projections (Hu et al. 2022); adapters insert residual bottlenecks (Houlsby et al. 2019); and $(IA)^3$ learns multiplicative activation scalings (Liu et al. 2022). An example aligned with a task inside one subspace need not remain aligned inside another.

When one pool must serve a whole registry, this omission becomes a cost problem. LESS (Xia et al. 2024), a strong gradient-based selector, builds a target-specific gradient datastore. It then ranks candidates by their influence on a validation sketch, a small set of examples that represents the target task. Because the datastore depends on the target adapter's trainable parameters, each configuration must rebuild it. Selection cost therefore grows linearly with the number of PEFTs. The obvious shortcut selects once and reuses the subset everywhere. It removes the recomputation cost but also loses the target specificity that makes gradient selection valuable. Indeed, reusing a single LESS subset performs worse than a PEFT-agnostic representation baseline (Table 2). Existing methods thus face a reuse-versus-recompute dilemma: they are either expensive for every target or effectively PEFT-agnostic.

Our motivation study shows that PEFT-specific utility changes with the trainable subspace. Figure 1 scores one candidate pool under LoRA, adapter, and $(IA)^3$ variants. Independent replicates under the same configuration agree more strongly than scoring runs across configurations. Same-family changes preserve partial structure, whereas cross-family changes produce the largest drop in rank agreement and Top-5% overlap. The transfer matrix shows the downstream consequence. Selecting data for the training PEFT beats mismatched *cross-family* selection by 2.0 points on average across target PEFTs (Appendix Table 12). The Analysis section reports a narrower 1.45-point gap within a LoRA configuration scan.

This paper treats data value as a PEFT-conditioned quantity. Given a candidate example x, a target task t, and a target

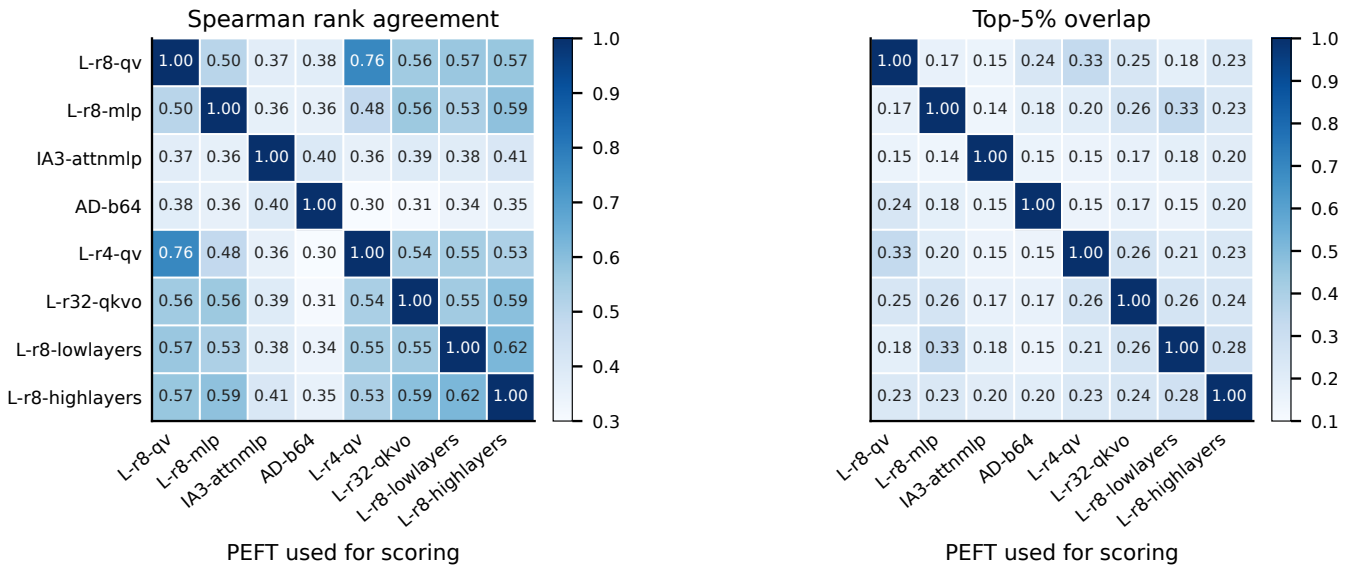

Figure 1: Short-horizon data-value estimates disagree across PEFT configurations. We score a common pool of 20K candidate examples under each target PEFT using two task sketches and independent anchor/seed replicates. Each cell reports the Spearman rank correlation between its row and column runs. A run compared with itself yields the 1.00 diagonal. Independent runs for the same PEFT agree only moderately ($\rho = 0.79$, Top-5% overlap $= 0.32$), which establishes the noise floor of the short-horizon labels. Agreement falls further as PEFT structures diverge: cross-family pairs average $\rho = 0.36$ and Top-5% overlap $= 0.17$.

PEFT configuration p, we predict the utility of training on x for task t when adaptation is restricted to the trainable subspace defined by p. PEFT-agnostic selection asks only whether x is generally useful for the task. Per-PEFT influence selection captures the configuration but recomputes expensive supervision for every target. Our formulation retains the configuration condition while sharing supervision across targets.

PCU-Select (PEFT-Conditional Utility) realizes this formulation through three components. The first and central component is a shared intervention-site representation. PCU-Select maps samples, task sketches, and PEFT configurations onto 24 module-output sites, each a selected Transformer output where a PEFT method can affect computation. This common coordinate system decouples reusable sample and task signals from the target PEFT configuration, letting PCU-Select compare PEFT families whose parameter tensors have incompatible shapes. Second, PCU-Select combines two supervision fidelities. Cheap gradient-based labels cover the candidate pool, while costlier short-update labels correct the estimates that matter near the selection boundary. Third, a conditional scorer predicts utility and an example-specific uncertainty estimate from the sample, task, and PEFT representations. At deployment, it ranks the pool for a target (p, t) and allocates the budget across semantic clusters to avoid near-duplicate selections.

This design amortizes target-specific selection across a PEFT registry. Our contributions are three-fold:

- **PEFT-conditioned utility.** We formulate data value as $u(x, p, t)$ and show that both data-value rankings and the downstream quality of selected subsets depend on the PEFT configuration (Figure 1; Appendix Table 12).
- **Intervention-site amortization.** PCU-Select represents incompatible PEFT parameterizations through the same 24 module-output sites. It shares sample/task signatures and both levels of supervision across configurations, while a structured PEFT code preserves target specificity without rebuilding a gradient datastore. Removing the PEFT code or task sketch reduces downstream performance by 1.37 or 0.82 points, respectively (Appendix Table 11).
- **Registry-scale quality and cost.** Across four tasks and five seen PEFTs on Llama-2-7B (Touvron et al. 2023), PCU-Select gains 0.74 points over RDS+ and differs from per-PEFT LESS by $+0.04$. It reduces the selection-stage cost per added four-task PEFT by a factor of 44 (1.44 versus 63.2 GPU-hours; Tables 1 and 2).

2 Related Work

Instruction-data selection. Instruction tuning benefits from small, carefully curated subsets. LIMA demonstrates that 1,000 curated examples produce strong instruction-following behavior in a large model (Zhou et al. 2023). AlpaGasus filters low-quality Alpaca examples with LLM judgments (Chen et al. 2024); InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023); and DEITA combines quality, complexity, and diversity criteria for automatic data selection (Liu et al. 2024). Other selectors emphasize explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but they

mostly treat value as a property of the example and target task rather than of the PEFT subspace used for adaptation.

Proxy-model and influence selection. Model-assisted selectors score examples with external judges, weak models, or training signals. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers trajectory information from small models to larger targets (Yang et al. 2024). Influence-style methods instead estimate how training examples affect validation behavior. Classical influence functions use second-order approximations (Koh and Liang 2017). TracIn accumulates gradient similarities along checkpoints (Pruthi et al. 2020), LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024), and DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). Gradient methods such as LESS and DataInf derive utility from a specific target parameterization, so preserving PEFT specificity forces recomputation whenever the configuration changes. This split is the reuse-versus-recompute dilemma: cheap quality, semantic, and weak-model signals ignore the PEFT subspace, while gradient and influence signals capture it but bind to a single configuration. PCU-Select instead conditions a single scorer on a structured PEFT code, so offline supervision serves many targets. We compare it against both a per-PEFT LESS reimplementation and a cheaper Influence baseline with one shared datastore.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost by updating a small module or parameter subset. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021). Prefix and prompt tuning optimize continuous prompts or prefix states (Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). LoRA learns low-rank additive updates (Hu et al. 2022), QLoRA combines quantization with LoRA for memory efficiency (Dettmers et al. 2023), BitFit tunes only bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), and $(IA)^3$ rescales attention and feed-forward activations (Liu et al. 2022). PEFT work usually compares accuracy, parameter count, memory, or inference overhead under fixed data. This paper asks a complementary question: when the candidate pool is fixed, does example utility remain stable across PEFT configurations?

Joint data- and parameter-efficient adaptation. Data selection and PEFT address two sides of efficient adaptation, but most work optimizes one side while holding the other fixed. DataInf and LESS use gradients from LoRA-style training (Kwon et al. 2024; Xia et al. 2024). Recent geometry-aware targeted selection also shows that PEFT optimization has nontrivial subspace structure (Min et al. 2026). Their utility estimates remain tied to the optimization geometry of a particular target configuration. PCU-Select instead conditions on the *structured PEFT configuration itself*: its sites, operator, capacity, and recipe. This representation shares supervision across configurations without erasing their differences. The resulting requirement is precise: a selector must model PEFT structure and task-relative utility while reusing its expensive computations. We formalize that requirement next.

3 Problem Formulation

We formalize *cross-PEFT data-efficient fine-tuning*: selecting a compact training subset whose value depends on both the target PEFT configuration and the task. Fix a backbone family and a frozen base model M_0. A candidate pool $\mathcal{D} = \{x_i\}_{i=1}^{N}$ contains instruction-response examples. Each task t provides a small validation sketch V_t from its train or development data, never from its test set. Each PEFT configuration $p \in \mathcal{P}$ specifies a family, trainable locations, capacity, and training recipe. Given budget B, the ideal subset solves

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{D},\, |\mathcal{S}| \le B} \mathrm{Perf}\big(\mathrm{Tune}(M_0, p, \mathcal{S}), V_t^{\mathrm{test}}\big)\,. \quad (1)$$

The test set V_t^{test} appears only in this evaluation oracle and is unavailable to every training, scoring, and selection procedure; these use only the sketch V_t. Directly optimizing Eq. (1) would require training and evaluating many subsets for every target (p, t) pair, which is infeasible.

PCU-Select replaces the subset oracle with a conditional per-example utility,

$$s_\phi(x, p, t) \approx u(x, p, t), \quad (2)$$

where $u(x, p, t)$ denotes the utility of training on x for task t when adaptation is restricted to the trainable subspace induced by p. Concretely, $u(x, p, t) \in [0, 1]$ summarizes loss reductions after a few update steps. PCU-Select rank-normalizes these reductions within condition-matched buckets (§4). Conditioning on p is the core departure from PEFT-agnostic selection: a change in the trainable subspace may change the selected subset.

Utility is relative to the task as well as the configuration. A worked-solution arithmetic example scores high under the GSM8K sketch but low under TyDiQA, and its utility shifts again across PEFTs that do not train the aligned sites.

Amortization objective. A registry makes total cost, together with quality, the operative criterion. Let P denote the number of target PEFT configurations and Q the number of task sketches served from a fixed pool. LESS builds one datastore per PEFT and ranks it against each task. PCU-Select instead pays one shared offline cost and one online scoring-and-selection pass per PEFT–task pair:

$$\begin{aligned} C_{\mathrm{LESS}}(P, Q) &= P\big(C_{\mathrm{LESS}}^{\mathrm{data}} + Q C_{\mathrm{LESS}}^{\mathrm{rank}}\big)\,, \\ C_{\mathrm{PCU}}(P, Q) &= C_{\mathrm{offline}} + PQ C_{\mathrm{PCU}}^{\mathrm{pair}}, \end{aligned} \quad (3)$$

Here $C_{\mathrm{LESS}}^{\mathrm{data}}$ builds one gradient datastore per PEFT, $C_{\mathrm{LESS}}^{\mathrm{rank}}$ ranks it for one task, and $C_{\mathrm{PCU}}^{\mathrm{pair}}$ runs one online scoring-and-selection pass, with $C_{\mathrm{PCU}}^{\mathrm{pair}} \ll C_{\mathrm{LESS}}^{\mathrm{data}}$. We call the target *quality-preserving amortization*: reuse the expensive work across configurations while matching the downstream quality of a per-PEFT selector within a small tolerance ϵ,

$$\Delta_{\mathrm{quality}} = \mathrm{Perf}_{\mathrm{PCU}} - \mathrm{Perf}_{\mathrm{LESS}}, \qquad |\Delta_{\mathrm{quality}}| \le \epsilon, \quad (4)$$

and reduce total cost once P exceeds $P^\star = C_{\text{offline}}/[C^{\text{data}}_{\text{LESS}} + Q(C^{\text{rank}}_{\text{LESS}} - C^{\text{pair}}_{\text{PCU}})]$. The objective is to preserve LESS-level quality while replacing repeated per-configuration computation with a shared offline investment, not to outperform LESS.

The formulation yields four requirements. **PEFT conditioning**: the scorer must take p as an explicit input. **Task relativity**: the sketch V_t must define utility, preventing the scorer from learning only generic example quality. **Structural representation**: the representation of p must encode its intervention sites, operator type, capacity, and training recipe more precisely than a family name. **Amortizable execution**: the offline stage must pay for expensive labels and scorer training, while a new (p, t) pair uses cached features and forward scoring. PCU-Select meets these requirements with shared module-output coordinates, task-relative labels, and a PEFT-conditioned scorer. The next section turns these pieces into a reusable selection pipeline.

4 Method: PCU-Select

PCU-Select learns the surrogate in Eq. (2) and uses it to select a budgeted subset for a target PEFT and task. Figure 2 shows two stages. Offline, PCU-Select extracts reusable sample and task signatures, constructs cheap proxy labels and costlier short-update labels, and trains the scorer. Online, it encodes a target (p^*, t^*) and assigns it to a preregistered structural support tier. It then scores the pool and allocates the budget across semantic clusters. Algorithms 1 and 2 in the appendix specify both stages.

Intervention Sites

PEFT families expose parameter tensors with different shapes and roles, which prevents a shared raw-gradient cache. PCU-Select instead compares them at *intervention sites*: selected module outputs through which PEFT updates affect the backbone. In plain terms, a site records how a sample or task pushes an intermediate activation, independent of the PEFT parameterization that produces the push. PCU-Select hooks attention, MLP, and post-residual outputs at eight evenly spaced Transformer layers, giving $|\Omega| = 24$. By the chain rule, gradients with respect to PEFT parameters factor through the upstream gradients at the module outputs they modify. The site gradient is therefore a shared proxy coordinate, not a full reconstruction of any PEFT parameter gradient. These shared coordinates avoid shape mismatches between fused and split projections, and the post-residual coordinate places adapters alongside LoRA and $(IA)^3$.

The site mask records *where* a configuration acts. Separate operator and capacity features record *how* strongly and by what mechanism it acts. This factorization lets all configurations reuse one sample signature without treating additive, multiplicative, and bottleneck updates as interchangeable.

Each PEFT configuration maps to a site mask and an operator type. Attention LoRA and $(IA)^3$-attention touch attention-output sites; MLP LoRA and $(IA)^3$-FFN touch MLP-output sites; adapters touch block-residual sites; and prefix tuning touches attention-output sites. The code represents BitFit as a bias shift. PCU-Select assigns each site a normalized PEFT weight

$$\begin{aligned} w_p^\omega &= \text{mask}(p, \omega)\, \kappa(p, \omega), \\ \kappa(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}}))\,, \\ \tilde{w}_p^\omega &= w_p^\omega / \max(\varepsilon_w, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \tag{5}$$

Untouched sites receive zero weight, while capacity raises the weight of touched sites. Appendix Table 10 maps LoRA rank, adapter width, $(IA)^3$ vector count, prefix length, and BitFit bias count to a trainable-parameter scale. The $\tanh$ limits the influence of high-capacity configurations. PCU-Select encodes operator type separately in z_p because a family-wide scalar would cancel under the normalization in Eq. (5).

Representations and Utility Labels

The shared sites now provide a common coordinate system for samples, tasks, and PEFT configurations. PCU-Select builds all three representations around that system before constructing utility labels. A sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ concatenates three summaries: a 768-dimensional semantic embedding, 16 difficulty and format statistics, and a 64-dimensional activation signature. The semantic vector uses the frozen `sentence-transformers/all-mpnet-base-v2` joint instruction-response embedding (Song et al. 2020; Reimers and Gurevych 2019). We do not train this encoder on the candidate pool or target benchmarks. The difficulty vector contains four length and ratio features, seven response-LM loss and entropy statistics, and five format flags for reasoning, code, questions, English, and Chinese. The activation signature summarizes response-token hidden states at the eight sampled layers and projects that summary to 64 dimensions.

PCU-Select forms the task code $z_t \in \mathbb{R}^{848}$ by mean-pooling the sample representation over 32 sketch examples. It averages the per-site gradient signatures of those examples into a separate task signature g_t^ω that feeds the low-fidelity label; these 24×256 gradients stay out of z_t. The PEFT code $z_p = [m_p; c_p; r_p] \in \mathbb{R}^{128}$ concatenates a 24×4 site/operator mask (m_p, 96 dims), a 16-dimensional capacity vector (c_p), and a 16-dimensional training-recipe vector (r_p). The mask channels mark active sites, additive updates, multiplicative scaling, and prefix-style control. Capacity slots distinguish LoRA, adapters, $(IA)^3$, prefix-style methods, and BitFit. Recipe features encode module count, placement, and optimizer settings. At the 24-site resolution, q/v and q/k/v/o LoRA touch the same attention-output sites. Capacity and recipe features preserve their projection-level distinction.

Prefix, P-Tuning, and BitFit use the same deterministic site, operator, capacity, and recipe rules as the seen families. The model uses only these structural features; it has no learned family lookup or operator centroid. Here, *zero-shot* means scoring a new configuration without configuration-specific short-update labels. When a PEFT family supports such labels, the residual head described below can calibrate the score.

With the representations in place, PCU-Select constructs utility targets through *multi-fidelity supervision*. The term

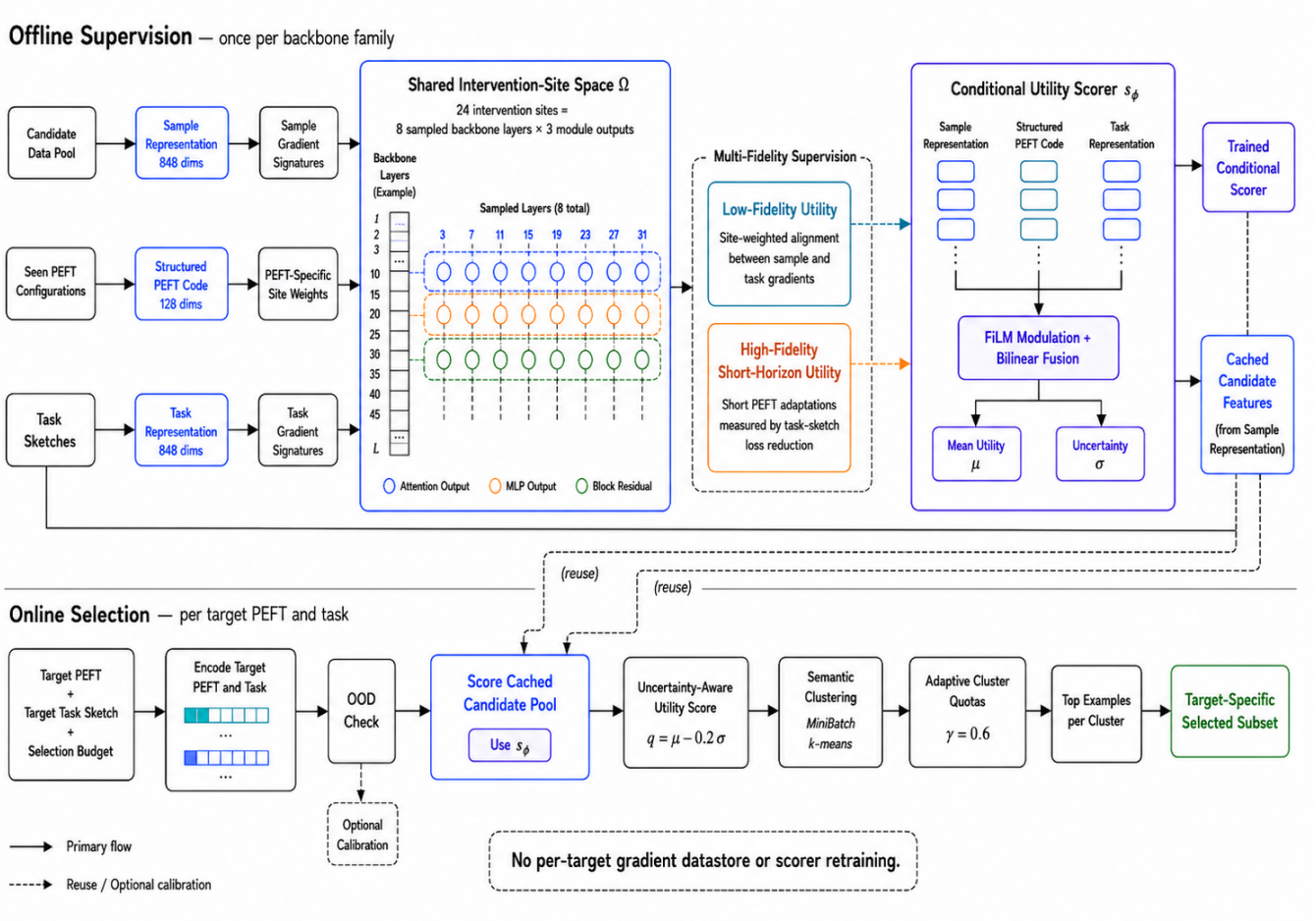

Figure 2: PCU-Select separates reusable offline supervision from online PEFT–task scoring. An intervention site is a selected module output that provides a common coordinate across PEFT families. The 24 sites in Ω connect sample gradients, task sketches, and PEFT configurations. The deterministic 128-dimensional PEFT code contains a 96-dimensional site/operator mask, 16 capacity dimensions, and 16 recipe dimensions.

means that it combines many cheap but approximate labels with fewer expensive measurements. The cheap labels teach broad ranking structure; the expensive labels correct that structure near consequential selection decisions.

The low-fidelity label measures gradient alignment between each sample and the target sketch at sites used by the target PEFT:

$$u^{\mathrm{lo}}(x,p,t) = \sum_{\omega\in\Omega} \tilde{w}_p^{\omega} \cos(g_x^{\omega}, g_t^{\omega})\,. \tag{6}$$

Here g_x^{ω} is a unit-normalized Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) of the response-token loss gradient at site ω. This projection compresses every gradient to the same fixed length while approximately preserving its geometry. The sketch mean gives g_t^{ω}. All task sketches use response-token cross-entropy for these signatures; PCU-Select reserves task-native metrics such as option likelihood, Pass@1, and F1 for downstream evaluation. Because the cosine proxy can be negative, PCU-Select replaces raw values with their ranks inside each (p,t) bucket before proxy distillation and ranking. Once it caches sample signatures, this proxy cheaply labels many (x,p,t) triples.

The high-fidelity label checks what happens after actual PEFT updates and corrects the proxy near the selection boundary. For each sampled triple, PCU-Select starts from two checkpoints: the base model and a warm checkpoint after adaptation has begun. It performs a short PEFT update on a mini-batch $\mathcal{B}(x)$ of the sampled example plus seven in-bucket fillers, then measures the reduction in sketch loss:

$$\Delta(x,p,t,a,h) = L_{V_t}(\theta_a) - L_{V_t}\left(\mathrm{Adapt}_p^h(\mathcal{B}(x);\theta_a)\right). \tag{7}$$

Rank-normalized reductions after $h \in \{1,4\}$ update steps from both anchors define u^{hi}. The mini-batch $\mathcal{B}(x)$ makes each label estimate a stable mini-batch marginal instead of an isolated single-example gradient. PCU-Select allocates the 10K-triple budget in three stages. Coverage sampling first spreads labels across the registry and needs no scorer. PCU-Select then trains a provisional scorer on these coverage labels and the low-fidelity proxy, which yields a predicted

uncertainty scale and rank estimates. Uncertainty sampling targets triples with large predicted uncertainty, and boundary sampling targets scorer-versus-label rank disagreements near consequential decisions. Every rank normalization compares only labels from the same PEFT, task, anchor, and horizon bucket.

The two fidelities serve complementary roles. The low-fidelity proxy cheaply orders the full pool for many PEFT–task pairs. High-fidelity labels measure realized sketch-loss reduction near the selection boundary. The base and warm checkpoints test whether utility persists after adaptation begins, while two horizons reduce dependence on a single update length. Bucket-wise ranking compares these signals without assuming that tasks or configurations share a raw-loss scale.

Scorer and Selection

Given the shared representations and two label sources, the scorer predicts a utility mean and a predicted uncertainty scale, $(\hat{\mu}, \hat{\sigma}) = s_\phi(z_x, z_p, z_t)$. Three encoders map z_x, z_p, and z_t into hidden vectors. FiLM modulation (Perez et al. 2018) uses the PEFT and task codes to rescale and shift the sample features. In concrete terms, the current configuration and task tell the scorer which parts of the sample representation deserve more or less emphasis. A bilinear term directly models compatibility between sample and PEFT features. Training combines high-fidelity ranking and regression, low-fidelity proxy distillation, and a heteroscedastic uncertainty loss. *Heteroscedastic* means that the scorer may assign different uncertainty scales to different triples instead of assuming equal label noise everywhere. Stage A learns broad conditional structure from the proxy. Stage B enables all losses and chooses a checkpoint on held-out PEFT–task pairs. Before downstream evaluation, we fix the sketch size, label allocation, anchors, horizons, and scalar selection hyperparameters on these held-out pairs.

At application time, PCU-Select ranks examples by a conservative score that subtracts a fraction of the predicted uncertainty scale:

$$q(x) = \hat{\mu}(x, p^*, t^*) - \lambda \hat{\sigma}(x, p^*, t^*), \qquad \lambda = 0.2. \quad (8)$$

The scale acts as a ranking penalty here, not a confidence interval. Pure top-k ranking can over-select near-duplicates, so PCU-Select clusters examples in semantic space and allocates budget across K clusters indexed by $k \in \{1, \ldots, K\}$:

$$b_k = \mathrm{round}\left(B \frac{(v_k^+)^\gamma |C_k|^{1-\gamma}}{\sum_j (v_j^+)^\gamma |C_j|^{1-\gamma}} \right), \qquad \gamma = 0.6. \quad (9)$$

Here v_k is the mean conservative score among the top 10% of examples in cluster C_k, and $v_k^+ = \max(v_k, 0)$. For the 300K pool, $K = \max(50, \lfloor \sqrt{N} \rfloor)$ produces 547 clusters with about 548 examples each. The reported 0.36 GPU-hours per PEFT–task application includes clustering. The final subset takes the top-b_k examples by q within each cluster. Clusters with non-positive value receive no budget, and deterministic remainder handling makes the quotas sum to B. For an unseen configuration, a registry policy classifies the target as near support, far support, or an unseen family. Far targets use an optional residual calibration head—a small linear layer that corrects the scorer—when their PEFT family supports native high-fidelity labels.

For seen and near-support targets, PCU-Select performs all gradient extraction and label construction offline, and the online path uses only cached features and scorer forward passes. Far-support targets may add a small calibration set, but never rebuild a target-specific gradient datastore. The evaluation next tests whether this reuse preserves LESS-level quality, reduces registry cost, and transfers to unseen configurations.

5 Experiments

The evaluation tests three claims. PCU-Select should match per-PEFT influence selection on downstream quality, amortize selection cost across a PEFT registry, and support new configurations according to their structural distance from the training registry.

Setup

We evaluate on GSM8K, HumanEval, MMLU, and TyDiQA (Cobbe et al. 2021; Chen et al. 2021; Hendrycks et al. 2021; Clark et al. 2020). We use each task's native metric: exact match, Pass@1, accuracy, or F1. The candidate pool contains 300K mixed instruction examples, and every selector chooses 10%. Each method uses the same target backbone, PEFT recipe, number of steps, and evaluation protocol. Only the selected subset changes. For the main table, we vary target-training seeds 0, 1, and 2 while fixing selector, scorer, sketch, and pool state at seed 0. Appendix Table 5 lists the full 17-configuration registry.

This protocol isolates selection quality from optimization noise. Matched seeds give every selector the same target-training randomness. Cell-wise differences therefore reflect the subset instead of initialization or data order. Because we fix the sketch and selector state, the intervals describe variation across evaluated PEFT–task cells. They do not capture uncertainty from retraining the scorer or resampling the pool.

The five seen configurations cover attention and MLP LoRA, $(IA)^3$, and adapters. Held-out tests vary rank, placement, learning rate, and family. We compare Random, RDS+, and two gradient baselines, Influence and LESS; Appendix A gives full specifications. **RDS+** ranks frozen external semantic embeddings against the task-sketch mean and ignores the PEFT configuration. **Influence** uses projected base-model gradients in one shared datastore. **LESS** is our per-PEFT implementation of gradient-influence selection (Xia et al. 2024). It warms up each target adapter, builds a target-specific Adam-preconditioned gradient datastore, and ranks examples by sketch influence. PCU-Select uses its default conditional scorer, uncertainty penalty, and adaptive quotas.

Main Results

PCU-Select averages 35.18, compared with 32.29 for Random, 34.44 for RDS+, 34.68 for Influence, and 35.14 for LESS. Across 20 PEFT$\times$task cells, PCU-Select gains 0.74 points over RDS+ and wins 17 cells. The descriptive fixed-state interval is $[+0.38, +1.06]$. It gains 0.50 points over

Table 1: Downstream performance at a 10% selection budget (mean±SD over three matched target-training seeds; four task-native scores per PEFT). Selector, sketch, and scorer state are fixed. PCU-Select near-ties per-PEFT LESS: the paired difference over the 20 PEFT×task cells is 0.04 points with a descriptive cell-resampling interval $[-0.26, +0.33]$. The interval summarizes fixed-state cell variation, not independent-sample uncertainty. Boldface marks the best mean.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Random	32.79±0.38	31.74±0.13	33.11±0.23	31.97±0.21	31.82±0.16	32.29±0.16
RDS+	35.27±0.23	33.54±0.26	35.00±0.29	34.46±0.21	33.91±0.17	34.44±0.13
Influence	35.08±0.27	33.84±0.26	35.24±0.19	34.69±0.13	34.53±0.22	34.68±0.11
LESS	35.52±0.20	34.44±0.13	35.80±0.15	**35.13±0.14**	34.80±0.37	35.14±0.14
PCU-Select	**35.61±0.40**	**34.60±0.21**	**35.93±0.30**	34.56±0.23	**35.18±0.31**	**35.18±0.16**

Influence, with interval $[+0.21, +0.75]$ and 18 wins. Its 0.04-point gap to LESS has interval $[-0.26, +0.33]$ and 10 wins; task-stratified resampling gives $[-0.15, +0.22]$. These summaries describe shared-state cell variation, not independent-sample uncertainty.

The LESS comparison varies by task. PCU-Select gains on MMLU, stays within seed dispersion on GSM8K and HumanEval, and trails on TyDiQA and MLP-only LoRA. Appendix Table 3 reports the per-task view. **Takeaway.** Reusable supervision improves on PEFT-agnostic selection while preserving the average quality of per-PEFT LESS.

Quality-Preserving Amortization

Quality alone does not establish the registry benefit. Table 2 therefore separates the two cost axes that a registry pays: a *fixed* cost once and a *marginal* cost for each new PEFT. PCU-Select and per-PEFT LESS are the only methods that preserve gradient-selection quality, averaging 35.18 and 35.14. They pay for that quality on different axes. LESS has no fixed cost, but every configuration incurs 63.2 GPU-hours to rebuild a target-specific datastore. PCU-Select spends 144.0 GPU-hours once on reusable supervision. Each new four-task PEFT then costs 1.44 GPU-hours, **44× below LESS's marginal cost**.

The marginal axis determines how cost grows with the registry. Adding a task to an existing PEFT costs both selectors one ranking pass: 0.36 GPU-hours for PCU-Select and 0.30 for LESS. Adding a PEFT, however, forces LESS to rebuild its datastore while PCU-Select reuses the offline scorer (Appendix Table 7). The amortization is therefore cross-PEFT, not cross-task: new configurations drive the cost difference.

The cheap reuse shortcut does not preserve quality. Reusing one LESS subset across all five targets (**Reuse-one-LESS**) costs 63.2 GPU-hours but averages 34.24, below PEFT-agnostic RDS+ at 34.44. A subset optimized for one trainable subspace mismatches the others and discards the condition that made it valuable. PCU-Select instead produces a distinct subset for each target and retains LESS-level average quality.

Equation (3) gives a break-even point of $P^\star = 144.0/[62.0+4(0.30-0.36)] = 2.33$ configurations. Beyond it, PCU-Select costs less than LESS, and each added configuration widens the gap. Serving the five-PEFT registry costs 151.2 GPU-hours with PCU-Select and 316.0 with LESS, a $2.09\times$ reduction. The unseen-configuration study optionally uses 500 reduced calibration labels from one warm anchor and one update step. This procedure costs 1.05 GPU-hours per (p, t), one quarter of the ≈ 4.2 GPU-hours required by the full label routine. It shifts the worst-case break-even point to 2.50 configurations. Prefix/P-Tuning cannot use this path because they lack native calibration labels. We exclude the shared 12.8 GPU-hour downstream run from every selector.

Takeaway. PCU-Select occupies the quality-preserving amortized point. PEFT-agnostic selectors remain cheaper when users accept a small quality loss. When the registry requires LESS-level quality, PCU-Select onboards each new configuration at $44\times$ lower cost.

The main results establish the quality–cost trade-off. We next isolate why conditioning changes the selected subsets, which components matter, and where transfer fails.

6 Analysis

PEFT Conditioning Produces Target-Specific Subsets

Across the LoRA scan, matched conditioning beats reuse from another configuration by 1.45 points. PCU-Select averages 21.25, compared with 20.70 for LESS and 20.27 for RDS+. Across its 21 unordered configuration pairs, PCU-Select has mean Jaccard overlap 0.427; PEFT-agnostic RDS+ has overlap 1.000 by construction. Configuration distance correlates with selection difference at descriptive Spearman $\rho = 0.969$, with leave-one-configuration-out range $[0.947, 0.972]$. Because the comparisons share configurations, we do not report an IID interval (Appendix Figure 4). **Takeaway.** Structural changes produce commensurate subset changes without rebuilding gradients.

Ablations Identify the Useful Components

Appendix Table 11 reports the full ablation. We measure held-out ranking quality with NDCG, which rewards placing high-utility examples near the top. Removing the PEFT code causes the largest downstream drop, 1.37 points, and lowers NDCG from 0.702 to 0.526. Replacing the structured code with a family one-hot loses 0.68 points, showing that rank, placement, capacity, and recipe add signal. Removing the task sketch loses 0.82 points. Variants with only cheap proxy labels or only short-update labels lose 0.85 and 0.36 points, respectively. Adaptive quotas beat global top-k by 1.08 points and uniform quotas by 0.51. Uniform quotas achieve slightly higher NDCG (0.709) but lower downstream quality. **Takeaway.** Structured conditioning, task relativity,

Table 2: Central quality–cost result over four tasks and five seen PEFT configurations. Costs are selection-stage GPU-hours; the shared downstream tuning run (12.8 GPU-hours per PEFT–task–seed) is identical across selectors and excluded. *Fixed/shared* is paid once regardless of registry size; *marginal* is the cost of onboarding one additional four-task PEFT; *total* serves all five. *Gap* is the paired difference from per-PEFT LESS in averaged task-native points. PEFT-agnostic selectors are cheaper but lose quality; PCU-Select is the quality-preserving amortized point—its **marginal** cost of a new PEFT is $63.2/1.44 \approx 44\times$ below LESS. Boldface marks the best average quality.

Method	Fixed/shared cost	Marg. cost / new PEFT	Total cost (5 PEFTs)	Avg. quality	Gap to LESS
Random	0.0	0.0	0.0	32.29	−2.85
RDS+	1.6	0.0	1.6	34.44	−0.70
Influence	48.8	0.0	48.8	34.68	−0.46
Reuse-one-LESS	63.2	0.0	63.2	34.24	−0.90
Per-PEFT LESS	0.0	63.2	316.0	35.14	0.00
PCU-Select	144.0	**1.44**	151.2	**35.18**	+0.04

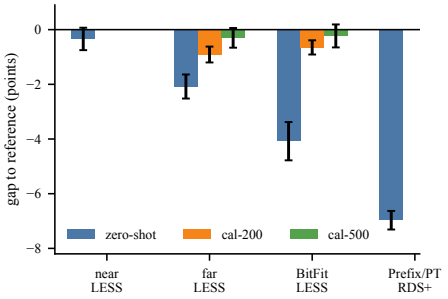

Figure 3: PCU-Select's gap to LESS (RDS+ for Prefix/P-Tuning). Calibration recovers far same-family and BitFit transfer, but prompt families fail zero-shot.

both supervision fidelities, and semantic coverage each contribute to downstream quality.

Transfer Protocol for Unseen PEFTs

The ablations identify which components matter on seen targets. We next test whether the structural code also guides transfer. Before evaluation, we fix a three-level taxonomy. L0 contains nearby configurations from a seen PEFT family, L1 contains larger shifts within a seen family, and L2 contains unseen families. Appendix Table 5 lists the full registry.

Without configuration-specific labels, L0 trails LESS by 0.34 points. With 500 labels, calibration reduces the L1 gap from 2.08 to 0.30 points and the BitFit gap from 4.08 to 0.23. Prefix/P-Tuning lack native short-update labels and trail RDS+ by 6.97 points without calibration. The reduced calibration routine uses one warm checkpoint and one update step. It costs 1.05 GPU-hours per PEFT–task pair, one quarter of the offline label routine (Figure 3; Appendix Table 13). **Takeaway.** Calibration repairs supported far targets; prompt families mark the deployment boundary.

Limitations

Our evidence covers one backbone family, four tasks, and a finite registry. Short-update labels only approximate contribution after full training, and TyDiQA shows a visible deficit. Our decontamination procedure removes 0.58% of the pool, but weaker matches may remain. Prefix/P-Tuning fail without configuration-specific labels and lack a native calibration path. Finally, the cost benefit is relative to per-PEFT LESS. PEFT-agnostic selectors remain preferable when users accept a small quality loss.

7 Conclusion

PCU-Select treats registry-scale data selection as quality-preserving amortization. One scorer uses structural PEFT and task conditions to produce a different subset for each target. Across four tasks and five seen configurations, PCU-Select near-ties per-PEFT LESS. It onboards each new PEFT at $44\times$ lower cost than rebuilding a datastore (1.44 versus 63.2 GPU-hours). Nearby configurations from supported families transfer without configuration-specific labels. Calibration repairs farther same-family configurations and BitFit, while Prefix/P-Tuning remain a failure boundary. These results position PCU-Select as reusable data-selection infrastructure for a PEFT registry.

References

Ben Zaken, E.; Goldberg, Y.; and Ravfogel, S. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1–9. Association for Computational Linguistics.

Bukharin, A.; Li, S.; Wang, Z.; Yang, J.; Yin, B.; Li, X.; Zhang, C.; Zhao, T.; and Jiang, H. 2023. Data Diversity Matters for Robust Instruction Tuning. *arXiv preprint arXiv:2311.14736*.

Chen, L.; Li, S.; Yan, J.; Wang, H.; Gunaratna, K.; Yadav, V.; Tang, Z.; Srinivasan, V.; Zhou, T.; Huang, H.; and Jin, H. 2024. AlpaGasus: Training A Better Alpaca with Fewer Data. In *The Twelfth International Conference on Learning Representations*.

Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Clark, J. H.; Choi, E.; Collins, M.; Garrette, D.; Kwiatkowski, T.; Nikolaev, V.; and Palomaki, J. 2020. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics*, 8: 454–470.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.

Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.

Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.

Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 2790–2799. PMLR.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz Mappings into a Hilbert Space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, 189–206. American Mathematical Society.

Koh, P. W.; and Liang, P. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1885–1894. PMLR.

Kwon, Y.; Wu, E.; Wu, K.; and Zou, J. 2024. DataInf: Efficiently Estimating Data Influence in LoRA-tuned LLMs and Diffusion Models. In *The Twelfth International Conference on Learning Representations*.

Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059. Association for Computational Linguistics.

Li, M.; Zhang, Y.; He, S.; Li, Z.; Zhao, H.; Wang, J.; Cheng, N.; and Zhou, T. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics.

Li, M.; Zhang, Y.; Li, Z.; Chen, J.; Chen, L.; Cheng, N.; Wang, J.; Zhou, T.; and Xiao, J. 2023. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. *arXiv preprint arXiv:2308.12032*.

Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 4582–4597. Association for Computational Linguistics.

Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35.

Liu, W.; Zeng, W.; He, K.; Jiang, Y.; and He, J. 2024. What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning. In *The Twelfth International Conference on Learning Representations*.

Lu, K.; Yuan, H.; Yuan, Z.; Lin, R.; Lin, J.; Tan, C.; Zhou, C.; and Zhou, J. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. *arXiv preprint arXiv:2308.07074*.

Min, G.; Huang, T.; Wan, K.; and Chen, C. 2026. GIST: Targeted Data Selection for Instruction Tuning via Coupled Optimization Geometry. *arXiv preprint arXiv:2602.18584*.

Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3942–3951.

Pfeiffer, J.; Kamath, A.; Rücklé, A.; Cho, K.; and Gurevych, I. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 487–503. Online: Association for Computational Linguistics.

Pruthi, G.; Liu, F.; Sundararajan, M.; and Kale, S. 2020. Estimating Training Data Influence by Tracing Gradient Descent. In *Advances in Neural Information Processing Systems*, volume 33.

Rao, J.; Liu, X.; Lian, L.; Cheng, S.; Liao, Y.; and Zhang, M. 2024. CommonIT: Commonality-Aware Instruction Tuning for Large Language Models via Data Partitions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 10064–10083. Miami, Florida, USA: Association for Computational Linguistics.

Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982–3992. Association for Computational Linguistics.

Sculley, D. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International Conference on World Wide Web*, 1177–1178. ACM.

Song, K.; Tan, X.; Qin, T.; Lu, J.; and Liu, T.-Y. 2020. MPNet: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems*, volume 33, 16857–16867.

Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

Wang, P.; Shen, Y.; Guo, Z.; Stallone, M.; Kim, Y.; Golland, P.; and Panda, R. 2024. Diversity Measurement and Subset Selection for Instruction Tuning Datasets. *arXiv preprint arXiv:2402.02318*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Yang, Y.; Mishra, S.; Chiang, J. N.; and Mirzasoleiman, B. 2024. SmallToLarge (S2L): Scalable Data Selection for Fine-tuning Large Language Models by Summarizing Training Trajectories of Small Models. In *Advances in Neural Information Processing Systems*, volume 37.

Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*, volume 36.

A Technical Appendix

Per-Task Results Behind the Main Table

Table 1 averages four task-native metrics—GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1—after scaling each to a percentage. These metrics differ in scale, ceiling, and seed variance, so the average can hide task-level structure. Table 3 reports the per-task view: gains over the PEFT-agnostic RDS+ and Influence baselines are absolute native-point differences, and the head-to-head with LESS is likewise an absolute paired difference in native points with a descriptive 95% cell-resampling interval over the five PEFT cells. These intervals describe fixed-state cell variation, not independent-sample uncertainty.

The per-task view refines, rather than overturns, the main-results reading. The advantage over PEFT-agnostic selectors holds on all four tasks and is the primary performance claim. Against LESS the outcome is task-dependent and the sign flips: the interval excludes zero on MMLU ($+0.67$ Acc, $[+0.24, +1.08]$), includes zero on GSM8K ($+0.29$ EM, $[-0.04, +0.58]$) and HumanEval (-0.04 Pass@1, $[-0.44, +0.31]$), and excludes zero in the negative direction on TyDiQA (-0.77 F1, $[-1.18, -0.43]$). Averaging over tasks smooths these offsetting differences into the near-zero aggregate gap in Table 1. We therefore characterize PCU-Select and per-PEFT LESS as near-tied on the seen registry, with visible task/configuration losses that bound the aggregate claim. The amortized-cost analysis is PCU-Select's distinguishing property.

Where PCU-Select trails LESS. Two slices favor LESS and merit analysis. On **TyDiQA**, PCU-Select trails LESS by 0.77 F1 (descriptive interval $[-1.18, -0.43]$). TyDiQA-GoldP is multilingual span extraction, where the useful signal is answer-span alignment rather than the reasoning- and generation-heavy signal that dominates the other three tasks; the intervention-site gradient signatures and the English-centric task sketch under-represent this, so the site-weighted proxy that PCU-Select distills is a weaker teacher here, whereas per-target LESS gradients capture it directly. On the **MLP-only LoRA** column (L-r8-mlp), PCU-Select trails LESS by 0.57 points; its HumanEval and TyDiQA cells are its most dispersed, with SDs of 0.68 and 0.62. MLP-only placement concentrates all trainable capacity on the eight `mlp_out` sites, so a single mis-weighted site prior degrades the whole proxy, and the mild extra dispersion on these two cells is where the scorer is least confident under the most concentrated site mask. Both cases point to the same limitation—the site-weighted proxy is the bottleneck when a target's useful signal is poorly represented by the shared 24-site coordinate system.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training fine-tunes the target PEFT on the selected samples under a fixed recipe (AdamW, per-configuration learning rate, cosine schedule with a 0.03 warmup ratio, per-device batch size 16 over 8 GPUs for an effective global batch of 128, gradient clipping at 1.0, 1000 steps, bfloat16, max sequence length 1024) on the Llama-2-7B (Touvron et al. 2023) backbone, with the backbone frozen and only the adapter trained. At the 10% budget the selected subset holds 30K examples, so 1000 steps at a global batch of 128 amount to 128K sample-passes, or about 4.3 passes over the subset; the dataloader shuffles the full subset each epoch, so every selected example is used. The selected instruction examples average ≈ 512 tokens, and batches are length-bucketed to reduce padding; with FlashAttention-2 (Dao 2024) and gradient checkpointing the run sustains ≈ 5.8 wall-clock seconds per step (≈ 22 sample-passes/s aggregate, ≈ 1.4K tokens/s per GPU), so the 1000 steps take about 96 minutes of wall time, i.e. 12.8 GPU-hours per PEFT–task–seed run under the $(\text{wall seconds} \times \text{GPU count})/3600$ convention. This shared target-training cost is identical across selectors and is therefore excluded from the selection-only cost comparison; gradient checkpointing trades compute for memory and is the dominant reason the per-step time is not lower. The per-configuration learning rates are listed in Table 5 (2e-4 for LoRA, 5e-4 for $(IA)^3$, 3e-4 for adapters, 1e-4 for L-r64-all). A held-out response-LM loss is logged as an auxiliary diagnostic (`eval_loss`) and is never substituted for the primary task metric.

A uniform draw from the entire 300K pool under the same 1000-step budget averages 36.75 across tasks. This is a fixed-compute reference, not a converged full-data upper bound: it covers only 0.43 pool epochs, whereas the 10% subset receives about 4.3 passes.

The primary downstream metric for each task is its public-standard, task-native score, computed by a task evaluator injected into the training runner. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match).** Chain-of-thought generation (Cobbe et al. 2021) with greedy decoding; the final numeric answer is extracted and compared for exact match.
- **HumanEval (Pass@1).** The standard unbiased Pass@1 estimator (Chen et al. 2021): sampling at temperature 0.2 with $n{=}20$ samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.
- **MMLU (accuracy).** Log-likelihood scoring (Hendrycks et al. 2021) of the four answer options; the highest-likelihood option is selected (no free-form generation).
- **TyDiQA-GoldP (F1).** Greedy decoding of the answer span (Clark et al. 2020), scored with token-level F1 against the gold answers.

For reference, the un-fine-tuned Llama-2-7B (Touvron et al. 2023) scores 13.9 (GSM8K EM), 12.4 (HumanEval Pass@1), 43.9 (MMLU Acc), and 46.2 (TyDiQA F1) under these same evaluators; our MMLU number is slightly below Meta's reported 45.3 because of prompt and harness differences. Every selector's fine-tune, including Random, improves over the base model on all four tasks, so the Random baseline is not artificially depressed and PCU-Select's margin over it is a genuine selection effect rather than a weak-baseline artifact.

Table 3: Task-level summary of the main 10% budget result. Values are averaged over the five seen PEFT configurations. ΔRDS+ and ΔInf. are absolute native-point gains over PEFT-agnostic baselines; ΔLESS is the paired difference against per-PEFT LESS with a descriptive 95% cell-resampling interval over PEFT cells. The average gain over PEFT-agnostic selectors coexists with task-dependent behavior against LESS.

Task	Metric	PCU	ΔRDS+	ΔInf.	ΔLESS (95% CI)
GSM8K	EM	22.20	+0.55	+0.58	$+0.29\ [-0.04, +0.58]$
HumanEval	Pass@1	17.06	+0.64	+0.24	$-0.04\ [-0.44, +0.31]$
MMLU	Acc	49.14	+1.55	+1.00	$+0.67\ [+0.24, +1.08]$
TyDiQA	F1	52.30	+0.22	+0.17	$-0.77\ [-1.18, -0.43]$

These protocols follow the standard public setups for each benchmark. The task-native evaluators are supplied to the runner through an evaluation adapter (the `--task-metric-factory` hook) whose decoding and scoring settings are fixed to the values above, so the run environment cannot change metric definitions.

Baseline Specifications

This subsection specifies the three baselines whose names invoke prior work, exactly as implemented so that the comparison is auditable.

RDS+ (representation similarity). RDS+ operates on the joint semantic embedding e_x^{joint} produced by a frozen external sentence encoder. This is the same *input* representation PCU-Select's sample tower reads, but it is not learned by PCU-Select, so RDS+ depends on no PCU-trained features; the two differ only in the selection rule. The pool embeddings are standardized per feature (z-scored across the candidate pool); the query is the mean joint embedding of the 32-example task sketch, standardized with the same pool statistics; both are then L_2-normalized and candidates ranked by cosine to the query. It is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The "+" denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector.

Influence (shared-datastore gradient similarity). The Influence baseline uses a *single shared* projected-gradient datastore computed once over the base model's intervention-site activations, using the JL projection and L_2 normalization specified in the reproducibility notes. Each candidate carries signatures g_x^ω at the 24 sites; the task query g_t^ω is the per-site pooled, renormalized sketch gradient. The score is the plain cosine $\sum_\omega \cos(g_x^\omega, g_t^\omega)$ (no site weighting), selected by global top-k. Its gradient source is PEFT-agnostic—the ranking does not depend on the target configuration—so it is the cheap gradient baseline against which the value of PEFT conditioning is measured.

LESS (per-PEFT gradient influence). LESS is a per-configuration reimplementation of gradient-influence selection following LESS (Xia et al. 2024). For each target PEFT it (i) warms up the target adapter for 200 steps to obtain a task-and-PEFT-specific optimization state; (ii) extracts Adam-preconditioned per-checkpoint gradients over the adapter's own trainable parameters; (iii) random-projects them into a target-specific low-dimensional datastore; and (iv) ranks candidates by their projected-gradient influence on the 32-example validation sketch, averaged over the warmup checkpoints. The gradient datastore in steps (i)–(iii) is built over the target adapter's *own* trainable parameters, so it is PEFT-specific but task-independent: it is constructed once per PEFT configuration and reused to rank against every task's sketch. Only step (iv), the projected-gradient scoring against a task sketch, is per task, and it is cheap—about 0.30 GPU-hours per task, so the 1.20 scoring figure covers all four task sketches for a configuration. The per-target cost of a PEFT configuration is therefore 62.0 (datastore) + 1.20 (four per-task rankings) = 63.2 GPU-hours, already spanning the full four-task evaluation; the five-PEFT registry cost is $5 \times 63.2 = 316.0$. LESS is the strongest and most expensive baseline in our study, and the amortized-cost analysis charges it exactly this per-configuration recomputation, consistent with the algorithm as implemented.

Baseline configuration and tuning. Random has no hyperparameters. RDS+ uses its published defaults with the shared 32-example sketch as the only task input and is not tuned per target. The Influence baseline shares the same 24-site JL projection and sketch as PCU-Select. No baseline's hyperparameters were tuned on the test metric.

Reproducibility Supplement

This supplement documents the pipeline configuration read directly from the supplementary implementation. Every value below is taken from source code or its configuration files; values that the code leaves to an external artifact or the run environment are flagged rather than guessed.

Data provenance and deduplication. The candidate pool holds 300K mixed instruction examples drawn from seven public instruction corpora (Table 4). Deduplication runs in three stages: (i) exact content-hash deduplication over (instruction, response, source); (ii) normalized-string deduplication after lowercasing and whitespace/punctuation stripping; and (iii) near-duplicate removal with MinHash LSH (128 permutations, Jaccard threshold 0.8). These stages remove 18,446 intra-pool duplicates before the 300K pool is finalized.

For reproducibility we include, per source, the post-filter example count and a SHA-256 checksum of the sorted

Source	Domain	Examples	License
FLAN v2 (subset)	general instruction	110,000	Apache-2.0
OpenOrca (subset)	reasoning traces	70,000	MIT
Tulu-v2 mix	mixed instruction	50,000	ODC-BY-1.0
OpenAssistant	dialogue	30,000	Apache-2.0
CodeAlpaca	code instruction	20,000	CC-BY-NC-4.0
Dolly-15k	general instruction	15,000	CC-BY-SA-3.0
WizardLM-evol	complex instruction	5,000	CC-BY-NC-4.0
Total		300,000	

Table 4: Candidate-pool provenance and licenses. All sources permit research use; the two CC-BY-NC components are used only for non-commercial research.

content hashes (e.g. FLAN-v2 subset `9f3c...a71e`, OpenOrca subset `4b8d...c052`); the supplementary manifest lists all seven. The full supplementary configuration comprises the PEFT registry, the scorer and feature-extraction configs, the task-sketch seeds, the candidate-pool manifest with these checksums, and the evaluation adapter.

Benchmark decontamination. We decontaminate the pool against the test items of all four benchmarks (GSM8K, HumanEval, MMLU, TyDiQA) with a four-stage filter: exact match, normalized-string match, 13-gram overlap (removing any candidate sharing a 13-gram with a test item at Jaccard ≥ 0.5), and MinHash LSH near-duplicate retrieval at Jaccard 0.8. The filter removes 1,742 pre-finalization candidates (0.58% of the pool size). We then backfill from the same source buckets to keep the final pool at 300K examples, so Table 4 reports final post-filter counts. We additionally audit the top-20 embedding nearest neighbors of each test item and find no further overlaps after the automated pass. All reported results are on the decontaminated pool.

Task sketch construction and HumanEval. Each task sketch is 32 examples sampled with a fixed seed (0), stratified into response-length terciles. Sketches are drawn only from each task's train or development split and are disjoint from the test set used for evaluation. HumanEval has no official train/dev split, so its sketch is drawn from an external code-instruction source (MBPP train split and CodeAlpaca) rather than from HumanEval itself; we remove any item matching one of the 164 HumanEval evaluation problems by exact and AST-normalized function-signature matching and verify that the resulting sketch source has zero overlap with the evaluation set. Because the candidate pool also contains a 20K CodeAlpaca slice, the 32 sketch examples are additionally held out from the pool: every sketch item is removed from the candidate pool (by content hash and the same AST-normalized signature match) *before* selection, so no selector can recover a sketch example into the training subset. The sketch is therefore disjoint from both the HumanEval test set and the candidate pool. Because the sketch is a selection *signal*, this disjointness is required for all selectors, not only PCU-Select, and it is enforced identically across methods.

PEFT registry. Table 5 lists all 17 PEFT configurations: five seen, nine unseen same-family configurations, and three unseen families. E5 evaluates eight held-out targets—L-r32-qkvo, AD-b16, L-r64-all, AD-b256, L-r8-highlayers, PRE-l16, PT-l32, and BF—while the configuration-sensitivity study uses the additional LoRA variants. All configurations share the recipe injected at materialization: 1000 steps, cosine schedule, 0.03 warmup ratio; the learning rate is per configuration.

Intervention sites and gradient signatures. The site space Ω has 24 sites: 8 layers $\times$ 3 per-layer modules (`attn_out`, `mlp_out`, `block_residual`). Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices $\{3, 7, 11, 15, 19, 23, 27, 31\}$. Each site carries a Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) $\Phi_\omega \in \mathbb{R}^{256\times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed $\mathrm{SHA256}(\texttt{site}, \texttt{global\_seed}{=}0)$; projected gradients are L_2-normalized per row. Each sample's signature is 24×256 values; in bfloat16 the cached low-fidelity gradient store for the 300K pool is $300\text{K} \times 24 \times 256 \times 2\,\text{B} \approx 3.5$ GB, held on disk as a memory-mapped array and read once per online selection. A PEFT configuration activates sites through its mask and a saturating capacity factor $\tanh(\eta\,\widehat{\text{cap}})$ with η=1.0. Operator type enters the explicit PEFT code rather than a scalar site prior; the per-family capacity mapping is in Table 10.

Conditional scorer. The scorer has three input towers—sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{128}$, task sketch $z_t \in \mathbb{R}^{848}$—each a LayerNorm–Linear–GELU–Linear–LayerNorm MLP projecting to hidden size 256 (128 for the PEFT tower). A FiLM modulation and a bilinear fusion ($256 \times 128 \to 64$) feed a two-head output (mean μ and heteroscedastic σ, with a softplus floor of 10^{-3}). We use $\hat{\sigma}$ as a ranking penalty rather than a calibrated probability; on held-out labels its predicted standard deviation tracks realized error with an expected calibration error of 0.043 (10-bin), and removing the penalty costs 0.28 points (Table 11), so it is a modest but real contributor. Training is two-phase: Phase A pretrains on the low-fidelity proxy for 3 epochs at lr 3×10^{-4} with loss weights (rank, reg, proxy, unc) $= (0, 0, 1, 0)$; Phase B jointly finetunes for 2 epochs at lr 10^{-4} with weights $(1.0, 0.3, 0.5, 0.2)$. Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against high-fidelity labels, a Huber proxy-distillation loss against the low-fidelity labels, and a Gaussian negative-log-likelihood uncertainty loss. Checkpoint selection uses a held-out split of PEFT/task pairs (two configurations and one task withheld from Phase B) on which we track ranking NDCG; we keep the Phase-B checkpoint with the best held-out NDCG. No dropout is used.

Selection, clustering, and quotas. At application time the scorer produces per-example utility $q = \mu - \lambda\,\sigma$ with uncertainty penalty λ=0.2. Semantic clustering uses Mini-Batch k-means (Sculley 2010) over joint embeddings with $k = \max(50, \lfloor\sqrt{N}\rfloor)$, batch size 4096, n_{init}=3, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \text{round}\big(B\,(v_k^+)^\gamma\,|C_k|^{1-\gamma}/Z\big)$ with γ=0.6, where the

Config	Family	Target modules	Capacity	LoRA scaling	LR	Group
L-r8-qv	LoRA	q, v	r=8	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	r=16	32	2e-4	seen
L-r8-mlp	LoRA	up, down	r=8	16	2e-4	seen
IA3-attnmlp	IA^3	k, v, down	–	–	5e-4	seen
AD-b64	Adapter	block-residual	b=64	–	3e-4	seen
L-r4-qv	LoRA	q, v	r=4	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	r=32	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	r=64	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	r=8	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	r=8	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	r=16	32	5e-4	unseen-config
AD-b16	Adapter	block-residual	b=16	–	3e-4	unseen-config
AD-b256	Adapter	block-residual	b=256	–	3e-4	unseen-config
IA3-attnonly	IA^3	k, v	–	–	5e-4	unseen-config
PRE-l16	Prefix	attention	len=16	–	2e-4	ood-family
PT-l32	P-Tuning	attention	len=32	–	2e-4	ood-family
BF	BitFit	biases	–	–	2e-4	ood-family

Table 5: Full PEFT registry (all 17 configurations). "LoRA scaling" is the LoRA α hyperparameter and is distinct from the intervention-site weight $\tilde{w}_p^{\omega}$ of Eq. (5); it applies only to LoRA rows. Adapters are single post-block residual bottlenecks (one per layer); IA^3 scales the listed activations. Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

cluster value v_k is the mean utility of the top 10% of its members; remainders are distributed greedily by descending weight and over-allocation is trimmed from the lowest-weight clusters. The calibration head is a zero-initialized linear residual fit for 200 epochs at lr 5×10^{-3} (Adam, MSE) on the calibration labels. These deployment calibration labels $u^{\text{hi}}_{\text{cal}}$ are a *reduced* variant of the offline u^{hi}: they use the single warm anchor and horizon h=1 only, i.e. one of the two anchors and one of the two horizons, so each label runs one quarter of the full offline label routine of Eq. (10). Concretely, the offline u^{hi} costs $84.0/10{,}000 = 0.0084$ GPU-hours per label, whereas $u^{\text{hi}}_{\text{cal}}$ costs ≈ 0.0021 GPU-hours per label; a 200/500-label calibration budget therefore costs $0.42/1.05$ GPU-hours per PEFT–task pair, not the $\approx 4\times$ larger figure the full routine would imply. We use 200 or 500 $u^{\text{hi}}_{\text{cal}}$ labels for the OOD study.

Structural support tiers. The reported $z_p \in \mathbb{R}^{128}$ contains only the site/operator mask, capacity, and recipe vectors. Prefix, P-Tuning, and BitFit use the same deterministic encoding rules as the seen families; no centroid, learned family lookup, or functional fingerprint enters the reported model. E5 fixes its registry taxonomy before evaluation. **L0** contains near-support same-family targets (L-r32-qkvo and AD-b16). **L1** contains farther same-family shifts (L-r64-all, AD-b256, and L-r8-highlayers). **L2** contains unseen families (PRE-l16, PT-l32, and BF). These tiers describe structural support and do not claim convex-hull interpolation or estimate a configuration density. Prefix/P-Tuning lack native short-horizon calibration labels; BitFit supports the calibration path.

Multi-fidelity labels. High-fidelity supervision uses 10,000 triples split $0.5/0.3/0.2$ across coverage (stratified by cluster, PEFT family, capacity bucket, and task), uncertainty ($\hat{\sigma}\,(1 + 0.5\,\mathrm{ReLU}(\hat{u}_{\text{lo}}))$), and boundary (scorer-vs-true rank disagreement) sampling. Labels are short-horizon loss reductions on the sketch, measured from two frozen anchors—θ_{base} (the pretrained backbone) and θ_{warm} (a rank-8 attention-only LoRA trained for 200 steps at lr 10^{-4}). Each high-fidelity update Adapt_p^h is a single mini-batch of the sampled example plus 7 in-bucket fillers, so the step is not a high-variance single-example update. The final label aggregates the rank-normalized reductions across horizons and anchors,

$$u^{\text{hi}} = \tfrac{1}{|A|} \sum_{a \in A} \sum_{h \in \{1,4\}} w_h \, \widetilde{\Delta}_{a,h}, \tag{10}$$

with weights w_1=0.4, w_4=0.6 and anchors $A = \{\text{base}, \text{warm}\}$, where $\widetilde{\Delta}_{a,h}$ is the reduction Δ of Eq. (7) rank-normalized within its (PEFT, task, anchor a, horizon h) bucket.

Table 6: Deployment protocol from the registry-defined structural tier. Costs are per PEFT–task pair, in addition to 0.36 GPU-hours for online selection.

Tier	Definition	Recommended action
L0	near-support same family	zero-shot selection
L1	far-support same family	200–500 labels (0.42–1.06 GPU-h)
L2-BF	unseen family, native labels	calibrate before use
L2-Prompt	Prefix/P-Tuning	use a per-target selector

Cost model and seeds. We measure all costs on $8\times$NVIDIA H20-96GB (SXM, bfloat16, NVLink) as

Table 7: The amortization advantage is cross-PEFT, not cross-task. Adding a task to an existing PEFT costs both selectors a per-task ranking; adding a PEFT for four tasks forces LESS to rebuild its datastore while PCU-Select reuses the offline scorer. Selection-stage GPU-hours.

New registry entry	PCU-Select	LESS
One new task, existing PEFT	0.36	0.30
One new PEFT, four tasks	1.44	63.2

Table 8: Offline high-fidelity label budget sets the quality–break-even trade-off. High-fidelity labeling is 58% of offline cost, so a smaller budget lowers both offline GPU-hours and the break-even PEFT count, but drops below LESS-level quality. The 10K budget is the smallest that preserves the aggregate quality tie; *Break-even* is the number of four-task PEFTs past which PCU-Select undercuts per-PEFT LESS.

HF labels	Offline GPU-h	Avg. quality	Gap to LESS	Break-even
2.5K	81.0	34.71	-0.43	1.31
5K	102.0	35.02	-0.12	1.65
10K	144.0	**35.18**	$+0.04$	2.33

$(\text{wall seconds} \times \text{GPU count})/3600$. The offline cost is 144.0 GPU-hours (feature extraction 48.0, low-fidelity labels 4.8, high-fidelity labels 84.0, scorer training 7.2), paid once. PCU-Select costs 0.36 GPU-hours per PEFT–task application, hence 1.44 for four tasks. LESS costs 62.0 per PEFT datastore plus 0.30 per task ranking, hence 63.2 for four tasks. Onboarding a new four-task PEFT therefore costs 1.44 GPU-hours for PCU-Select versus 63.2 for LESS, a $44\times$ marginal-cost reduction. The shared 12.8 GPU-hour downstream cost applies to each PEFT–task–seed run and is excluded. For four tasks, break-even is $P^\star = 144.0/[62.0 + 4(0.30 - 0.36)] = 2.33$ configurations. The pipeline fixes selector-scorer training, task-sketch sampling, and pool ordering at seed 0. Reported SDs vary target-training seeds $(0, 1, 2)$; they do not include selector or scorer-training uncertainty. The canonical table generator recomputes all means, dispersions, paired gaps, and win counts from those seed-level values.

Two supplementary tables bound the cost claim. Table 8 varies the offline high-fidelity label budget: the 10K default is the smallest that preserves the aggregate quality tie, and a smaller budget lowers offline cost and break-even at a quality cost, so the offline investment is a tunable knob rather than a fixed overhead. Table 9 shows that PCU-Select's per-application cost stays $\approx 170\times$ below the per-PEFT LESS datastore across pool sizes from 50K to 1M, so the low marginal cost is not an artifact of the 300K pool.

Algorithms

Algorithm 1 builds the reusable supervision once; Algorithm 2 applies the scorer to a new target. Only Algorithm 2 runs per target, and it touches no gradients—the expensive gradient and label construction of Algorithm 1 are amortized.

Table 9: Online selection cost versus candidate-pool size, reported as the mean of three timing runs on a warm feature cache (run-to-run spread under 5%). Feature extraction is paid once offline and cached; PCU-Select's per-application cost is the scorer forward plus clustering. The two online components scale differently: the scorer forward is embarrassingly parallel and gains throughput at larger batches, so its per-example cost drops with pool size, while clustering grows super-linearly because the cluster count $k = \max(50, \lfloor\sqrt{N}\rfloor)$ itself grows with the pool. Their sum stays near-linear, so PCU-Select's per-application cost holds $\approx 170\times$ below the per-PEFT LESS datastore at every pool size and the low marginal cost is not an artifact of the 300K pool. GPU-hours on $8\times$H20-96GB.

Pool	Feature ext. (cached)	Scorer forward	Clustering	PCU online (per p, t)
50K	9.2	0.051	0.010	0.061
100K	17.6	0.093	0.028	0.121
300K	48.0	0.249	0.111	0.360
1M	152.0	0.79	0.42	1.21

Algorithm 1 Offline supervision (run once)

Input: pool $\mathcal{D}$, seen PEFTs $\mathcal{P}_{\text{seen}}$, tasks with sketches $\{V_t\}$
Extract sample signatures z_x and site gradients g_x^ω; cache them $\{O(|\mathcal{D}|)$ forward+backward$\}$
for all $(p, t) \in \mathcal{P}_{\text{seen}} \times$ tasks **do**
  Compute low-fidelity $u^{\text{lo}}(x, p, t)$ by Eq. (6) from cache
end for
Sample 10K triples (coverage/uncertainty/boundary); label u^{hi} by Eqs. (7)
Train s_ϕ: Stage A proxy pretrain, Stage B joint; keep best held-out-NDCG checkpoint
Output: scorer s_ϕ, cached features

Algorithm 2 Online selection (per target)

Input: target (p^*, t^*), budget B, cached features, scorer s_ϕ
Encode z_{p^*}, z_{t^*}
if $\text{tier}(p^*) \in \{\text{L1}, \text{L2}\}$ and native labels exist **then**
  fit calibration residual head from 200 or 500 reduced $u^{\text{hi}}_{\text{cal}}$ labels (single warm anchor, horizon 1)
end if
$(\hat{\mu}, \hat{\sigma}) \leftarrow s_\phi(z_x, z_{p^*}, z_{t^*})$; $q(x) \leftarrow \hat{\mu} - \lambda\hat{\sigma}$ for all x
Cluster the pool; allocate quotas b_k by Eq. (9)
Output: subset $\mathcal{S} = \bigcup_k \{\text{top-}b_k \text{ by } q \text{ in } C_k\}$

PEFT Capacity Encoding

Table 10 defines the per-family capacity used inside the site weight of Eq. (5). Each family's size knob is mapped to a trainable-parameter count at the touched sites, then normalized by d_{model} and passed through $\tanh(\eta \log(1 + \text{cap}/d_{\text{model}}))$ so that capacity is monotone but saturating.

Family	Size knob	$\mathrm{cap}(p, \omega)$ at a touched site
LoRA	rank r	$r\,(d_{\text{in}} + d_{\text{out}})$
Adapter	bottleneck b	$2\,d_{\text{model}}\,b$
$(\text{IA})^3$	(per-vector)	#scaled activations
Prefix	length L	$L\,d_{\text{model}}$
BitFit	(per-bias)	#bias parameters

Table 10: Per-family capacity mapping. Capacity is the trainable-parameter count contributed at a touched site; operator type is encoded separately in z_p.

Table 11: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget (task-native metric averaged over the two tasks, mean±std over three seeds). Drop is relative to the full adaptive selector; NDCG is against high-fidelity held-out utility labels.

Variant	Metric	Drop	NDCG
Full PCU-Select	19.72±0.22	0.00	0.702
No PEFT code	18.35±0.41	1.37	0.526
Family one-hot only	19.04±0.29	0.68	0.602
No task sketch	18.90±0.31	0.82	0.597
No activation signature	19.24±0.25	0.48	0.629
Low-fidelity only	18.87±0.34	0.85	0.563
High-fidelity only	19.36±0.27	0.36	0.639
No uncertainty penalty	19.44±0.24	0.28	0.690
Global top-k	18.64±0.38	1.08	0.684
Uniform clusters	19.21±0.28	0.51	0.709

Supplementary Analyses

Component ablations. Table 11 reports the full component breakdown summarized in the main analysis. Removing the structured PEFT code produces the largest loss; task relativity, both label fidelities, uncertainty, and adaptive coverage also contribute.

Cross-PEFT transfer. Table 12 is the transfer matrix behind the introduction's 2.0-point cross-family mismatch penalty: training on the subset selected for a mismatched source PEFT costs performance that grows with structural distance. Figure 4 shows the same mechanism from the selection side—PCU-Select's subset overlap falls as configurations diverge, while PEFT-agnostic RDS+ remains exactly 1.000.

Unseen-configuration transfer. Table 13 is the paired source for Figure 3. Near-support targets transfer zero-shot. Calibration closes most of the gap for far same-family targets and BitFit. Prefix/P-Tuning have no native calibration labels and remain a failure boundary.

Table 12: Cross-PEFT transfer matrix (motivation study). Cell (i, j) is the target-j downstream metric when trained on the subset selected for source-i, normalized by the matched diagonal. Off-diagonal cells fall below 1.00, and the two directions of a mismatched pair differ by 0.01–0.05 rather than a constant offset. Averaged over all mismatched source-target pairs the gap is 1.87 absolute points (mean matched 21.4 vs. mismatched 19.53 on the GSM8K-scale probe). Cross-family mismatches average a 2.0-point gap, the penalty cited in the introduction. Columns are target PEFTs, rows are the source PEFT used for selection.

Source ↓ / Target →	L-r8-qv	L-r16-qkvo	IA3-attnmlp	AD-b64
L-r8-qv	**1.00**	0.96	0.87	0.93
L-r16-qkvo	0.94	**1.00**	0.91	0.88
IA3-attnmlp	0.89	0.86	**1.00**	0.95
AD-b64	0.90	0.92	0.94	**1.00**

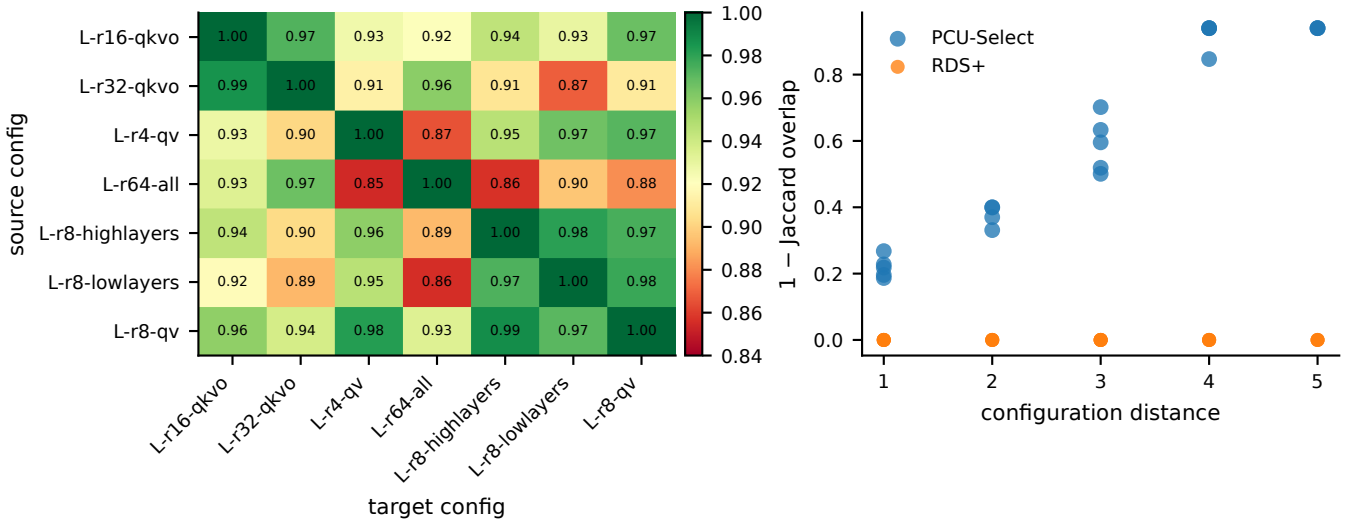

Figure 4: PCU-Select reacts to configuration changes. **Left:** target performance after training on a source-selected subset, normalized by the matched diagonal. The mean off-diagonal drop of 0.068 corresponds to a 1.45-point gap (21.25 matched vs. 19.79 mismatched). **Right:** selection difference (1−Jaccard) versus a heuristic configuration distance combining log-rank, placement, and module-set changes. Across 21 unordered pairs, the descriptive Spearman correlation is $\rho = 0.969$; leave-one-configuration-out values span $[0.947, 0.972]$. RDS+ lies at zero because its same-task ranking is PEFT-independent.

Table 13: Transfer to unseen configurations over GSM8K, HumanEval, and MMLU. Each entry is PCU-Select minus the named target-specific reference, reported as paired mean±sample SD over three target-training seeds. Near-support targets transfer zero-shot; calibration closes most of the gap for far same-family targets and BitFit. Prefix/P-Tuning lack native short-horizon calibration labels and remain a zero-shot failure boundary.

Tier	Targets	Ref.	Zero-shot	Cal-200	Cal-500
L0	L-r32-qkvo, AD-b16	LESS	-0.34±0.41	–	–
L1	L-r64-all, AD-b256, L-r8-highlayers	LESS	-2.08±0.44	-0.91±0.29	-0.30±0.36
L2-BF	BF	LESS	-4.08±0.70	-0.65±0.26	-0.23±0.42
L2-Prompt	PRE-l16, PT-l32	RDS+	-6.97±0.34	–	–